

这三个题难度差不多，但根据选手心理学，大多数人顺序开题，而 C 构造方案不好写，因此将 C 放到最后减少被喷。

而 B 部分分是 AGC 原题，也容易被喷。

因此把来历不明的 A 放到最前面最为合适。

A

考虑对于固定的 a 答案是什么：显然 $i \rightarrow a_i$ 这样的图构成内向基环树森林，先任选其中一颗基环树，设环长为 l ，对该基环树中每个点染一个颜色 $c_u \in [0, l)$ 使得模 l 意义下 $c_u = c_{a_u} + 1$ 恒成立。那么从 u 出发的人走 t 步之后走到点的颜色必定是 $(c_u - t) \bmod l$ ，且可以注意到充分长时间后，所有人都会聚集在环上，从而颜色相同的人最终都会聚集到一起。答案就是这颗基环树上所有颜色出现次数最大值，当然一开始可以任取基环树，因此外面再套一层 $\max$ 。

还要求任意修改至多 k 条出边后答案的最大值，那么修改至多 k 条出边，至多能将原图的 $k + 1$ 个连通块合并成新的弱连通块，并且有以下重要观察：

- 任取 k 个原图的弱连通块，答案可以取到这 k 个连通块大小之和。

证明：任取这 k 个连通块之一，设其环上有某个点编号为 u ，将 u 的出边改为自己，并将剩下 $k - 1$ 个连通块每一个选一个环上的点将出边改为 u 。容易验证这样将 k 个连通块合并成了一个环长为 1 的基环树。

从而至少能让答案取到原图前 k 大的连通块大小之和，要想比这个更大就只能是合并 $k + 1$ 个连通块的方案，此时必然是有一个连通块原封不动，而剩下 k 个连通块每一个中选一个点修改一条出边。

设连通块 P 原封不动，其环长为 l ， $C_1 \sim C_k$ 每个修改一条出边。新方案要想不劣，必须修改后选取的基环树同时包含来自 $k + 1$ 个原本连通块的点，不难发现只能是 $C_1 \sim C_k$ 都有点在修改后可一路走到 P 内点。

对每个 C_i ，枚举其修改了 u_i 的出边，则只有该连通块中原本可到达 u_i 的点可能计入最终的贡献（这里是针对可能修改了一条原本的树边来考虑，此时一些点可能原本无法抵达 u_i ，虽然直觉上这就是不优的），对这部分点按照模 l 构造染色，则这部分点被计入贡献的上界是这部分点中颜色出现次数最大值。

另一方面该上界是可取到的：考虑一个简单的策略是对每个 C_i 直接将 u_i 的出边指向连通块 P 环上的某个点，通过调整指向这 l 个点之一，可以使得来自 C_i 的这些点的染色整体模 l 意义下平移，从而完成“对齐”使得颜色出现次数在加和后的最大值，就等于每一项最大值再加和起来。

由此可见 u_i 必然取 C_i 某个环上点才是不劣的，现在可写出以下算法：

- 枚举原图某个连通块 P 原封不动，并计算 $a(P)$ 为对其染色后每种颜色出现次数最大值。
- 对原图其余连通块 C ，计算 $b(C)$ 为其任意断开一条环上边后，按照模 l 进行染色，每种颜色出现次数最大值。
- $\forall 0 \leq k \leq n$ ，将 ans_k 向 $a(P)$ 加上前 k 大的 $b(C)$ 之和取 $\max$ 。

直接实现可能需要 $\Theta(n^3)$ 时间，没有对应的子任务但是写题解的时候来不及改了，谢罪。

考虑如何高效实现第二步：设连通块 C 环上顶点依次为 $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$ ，若断开 $v_k \rightarrow v_1$ 的边，则一种可行染色是 v_i 子树内距离 v_i 为 d 的点染成 $(k - i + d) \bmod l$ （即染色方案是记录走到 v_k 所需步数）。而如果换成断开 $v_1 \rightarrow v_2$ 的边，改成记录走到 v_1 所需步数，挂在 $v_2 \sim v_k$ 子树上的点的颜色都是模 l 意义下加 1，而只有挂在 v_1 子树上的点由染成 $(k - 1 + d) \bmod l$ 变为 $d \bmod l$ ，对这部分重算即可。换言之可视作只对 v_1 子树内的点，其染的颜色在模意义下减去 k 。

问题转化为，进行 $\Theta(|C|)$ 次对一个序列的某一项 ± 1 ，时刻维护序列最大值，用桶记录序列中有多少项值为 $1, 2, \dots$ 即可，时间复杂度 $\Theta(|C|)$ 。

加上这个优化后只需要 $\Theta(n^2)$ 时间。

进一步优化的关键观察： l 只有 $O(\sqrt{n})$ 种，因为每个连通块环长之和至多为 n 。

于是不枚举 P 而是只枚举 P 对应的 l ，每次 $\Theta(n)$ 计算每个连通块在该 l 下的 a, b 权值。然后相当于需要对 $\forall 0 \leq k \leq n$ ，选出一个环长为 l 的连通块取其 a 再额外选出 k 个任意连通块取其 b ，将贡献加和，求最大贡献。

注意对于环长为 l 的连通块其 a, b 权值相等，所以视作选出 k 个连通块但包含至少一个环长为 l 的即可。只需一开始选上环长为 l 的中权值最大的，再每次多选一个权值最大的尚未选的。

时间复杂度 $\Theta(n\sqrt{n})$ 。

B

小故事：这题最早被计划用于 2024 北京市赛，然而比赛开始前我们发现 [怎么排列版本是个 AGC 原题](#)，而且难度怎么只有 *2000，当时我们觉得这题的分析很难。过了很久，它被我翻出来当作模拟赛题了。

我当时写的标程虽然也是 $\Theta(n^2)$ 的，但现在我完全看不懂是什么意思，不过顺着 AGC 原题官方题解的思路扩展到一般序列也是可以的。

首先可以认为操作是：选择 $1 \leq x, y \leq n, |x - y| = 1$ 且 $a_x \geq a_y$ ，将 a_y 赋值为 a_x 。

容易归纳说明，操作若干次之后 $\forall 1 \leq i \leq n, a_i$ 这个值要么会被复制到 b 的 $[l_i, r_i]$ 这段区间内，要么其根本不出现。

先考虑 a 是个排列怎么做。好消息是，在这个前提下，数合法区间列（定义为用上述操作可能得到的区间列）与数最终局面是等价的。

现在的目标：对于给出的一列 $[l_i, r_i]$ 要么满足 $l_i \leq r_i$ 要么 $[l_i, r_i] = \emptyset$ ，考虑如何判断这个区间列是否合法。

可以找到一个显然的限制：

1. 从左到右取出所有不为空的 $[l_i, r_i]$ ，这些区间构成一个对 $[1, n]$ 的划分（并起来为 $[1, n]$ 且两两无交，最重要的是区间顺序与下标顺序一致）。

此外，由于操作后所有位置的值都不减， $\forall 1 \leq i \leq n$ 找出最小的 L_i 使得 $a[L_i, i]$ 都 $\leq a_i$ 以及最大的 R_i 使得 $a[i, R_i]$ 都 $\leq a_i$ ，那么只有下标在 $[L_i, R_i]$ 内的值最终可能是 a_i ，这就得出了另一个限制：

2. 若 $[l_i, r_i] \neq \emptyset$ ，则 $L_i \leq l_i \leq r_i \leq R_i$ 。

关键结论：对于 a 是排列的部分分，对任意符合这两条限制的区间列，一定存在一种操作方案使得最终局面符合该区间列的描述。

证明：考虑归纳。找出 $a_p = n$ 是最大值出现的位置，容易说明对符合限制的区间列必有 $L_p \leq p \leq R_p$ 。简单的想法是对 $[1, p), (p, n]$ 两边分别构造出区间列描述的局面（但 $[L_p, R_p]$ 这些最终要求为 n 的位置是什么值无所谓）后再最后用操作把 $[L_p, R_p]$ 全都变为 n 。但递归到子问题时，例如考虑左侧， $[L_p, p)$ 最终值是什么无定义，即 $[1, p)$ 里的 $[l_i, r_i]$ 区间列只能并出 $[1, L_p)$ 因此不能直接由归纳法知可行。但事实上总能适当扩展区间将 $[L_p, p)$ 这些下标也添加进去并仍然得到符合两条限制的区间列（例如考虑 $L_p - 1$ 这个下标属于哪个值，然后让那个值拥有的区间继续往右边尽量扩展，直到无法扩展后设停止在 q ，则修改后 $[1, q]$ 的区间列是自洽的，而 (q, p) 这一段让所有 $l_i = r_i = i$ 就可以），这部分证明没有本质困难。

根据这个结论， a 是排列的部分分可以轻松解决：设 $dp_{i,j}$ 为考虑了 $1 \sim i$ 的区间该怎么选，且这些区间目前都符合限制且并为 $[1, j]$ （具体是 $L_i \leq l_i \leq r_i \leq R_i$ 以及无交、顺序合理）的方案数，直接转移是 $\Theta(n^3)$ 的，前缀和优化后变为 $\Theta(n^2)$ 。

对于 a 是任意序列，上述分析显然会算重，例如考虑 $n = 2, a = \{1, 1\}$ 则 $l_1 = l_2 = 1, r_1 = r_2 = 1$ ，正确答案是 1 而这样算出来会变成 3。问题的核心在于合法区间列和最终局面不再是双射，很多合法区间列其实对应同一局面。

考虑找代表元，对于一个 b ，只在**字典序最大**的对应到它的区间列处对这个局面计数。

字典序的定义：对于两组区间列 $\{(l_i, r_i)\}, \{(l'_i, r'_i)\}$ ，设最小的 $(l_i, r_i) \neq (l'_i, r'_i)$ 的下标是 p ，则：若其中有一个为空则空的那个序列字典序小，否则比较 r_i, r'_i （因为 l_i, l'_i 其实一定相同），较小的那个字典序小。

这样考虑是因为限制 1 和 2 可见，判断一个局面 b 是否存在至少一个对应到其的区间列（从而其是否可能作为最终局面）实际上可考虑以下贪心算法：

- 维护当前已有的区间的最大右端点是 cur ，初始 $cur = 0$ 。
- 从小到大考虑 $i \in [1, n]$ ，若 $L_i \leq cur + 1$ ，则取 r_i 为同时满足 $b(cur, r_i]$ 均为 a_i 且 $r_i \leq R_i$ 的最大值，然后将 cur 更新为 r_i 。
- 若最终 $cur = n$ ，可行，否则不行。

字典序最大的对应到 b 的区间列正是上述贪心算法构造的区间列，这一点的证明考虑一个区间列该如何同时满足限制 1 和 2 且对应局面 b 即可。

从而只需要对满足限制 1 和 2，且符合上述贪心原则（在同类中字典序最大）的区间列计数即可。

要符合贪心原则，必须符合这条限制：

3. 不能存在 $1 \leq i < j \leq n$ 使得 $a_i = a_j$ ，且 $[L_i, R_i] = [L_j, R_j]$ ，且 $\forall k \in (i, j), [l_k, r_k] = \emptyset$ ，且 $[l_j, r_j] \neq \emptyset$ 。

考虑若存在，总能将 $[l_j, r_j]$ 均添加到 $[l_i, r_i]$ 这里去，改完字典序更大。

关键结论：限制 3 完整描述了贪心原则。

证明：对于满足限制 1 和 2 但违背贪心原则的区间列 $\{(l_i, r_i)\}$ ，其既然并非字典序最大的，设同类中字典序最大的是 $\{(l'_i, r'_i)\}$ ，且 p 是首个不相同的位置，则要么 $[l_p, r_p] = \emptyset$ 要么 $r_p < r'_p$ ，但它们对应到相同的最终局面，因而对于 $\{(l_i, r_i)\}$ 而言，找出比 p 编号更大的首个非空区间 $[l_q, r_q]$ ，此时一定 $a_q = a_p$ ，取 $i = p, j = q$ 会违背限制 3。略显不直观的可能是此时必然 $[L_p, R_p] = [L_q, R_q]$ ，但由于 $a_p = a_q$ 这其实只需要 $a(p, q)$ 中没有比 a_p 更大的元素，而若存在， $[L_p, R_p]$ 和 $[L_q, R_q]$ 就不可能包含公共元素 $r_p + 1$ （意思就是在 $[l_i, r_i]$ 的区间列中只考虑 $1 \sim p$ 这段前缀，首个未被覆盖的编号，根据上述分析其同时可被 $[L_p, R_p], [L_q, R_q]$ 覆盖）。

于是只需要计数符合限制 1~3 的区间列，添加限制 3 后，沿用此前的 DP，只需在转移时注意若 i 与其最靠后的同色前驱 a_{prev} 满足 $[L_{prev}, R_{prev}] = [L_i, R_i]$ ，则额外减去 $dp_{prev,*}$ 的信息即可（容斥减掉 $(prev, i)$ 这些区间都为空，但 i 这个区间非空的方案数），时间复杂度仍为 $\Theta(n^2)$ 。

C

特判每一刀都沿着半径切的情况，这是平凡的。

否则可考虑问题 A：

- 阶段 1：花费 m 的代价，将 180 分为 $a_1 + a_2 \dots + a_m = 180$ ，每个 $a_i \geq 1$ ，代表所有沿直径的切割造成的影响。
- 阶段 2：一开始你拥有 $a_1 \sim a_m$ 各两个构成可重集 P 。每次可以选 $x \in P$ ，花费 1 的代价，选 $0 < y < x$ ，从 P 删掉一个 x 并添加一个 y 和一个 $x - y$ 。
- 以最小的代价和使得最终 P 包含 Q 作为子集。

先考虑 $\sum_{i=1}^n q_i = 360$ 的情况，此时最终目标是使得 $P = Q$ 。

此时可考虑问题 B：

- 初始下标集 $S = \{1, 2, \dots, n\}$, $A = B = \emptyset$ 。
- 进行 n 轮操作, 每轮选 $i \in S$, 然后将 i 从 S 删去, 并向 A 或 B 添加 i 。
- 结束时, 必须 $\sum_{i \in A} q_i \leq 180, \sum_{i \in B} q_i \leq 180$ 。
- 最大化前 $n - 1$ 轮操作后, $\sum_{i \in A} q_i = \sum_{i \in B} q_i$ 的次数, 设次数为 c , 可对问题 A 构造出代价为 $n - 1 - c$ 的方案。

最后一点是考虑, 先沿直径切一刀变为两个 180, 称为上下部分。向 A 添加 i 表示 q_i 这段弧属于上半部分, 向 B 添加则表示属于下半部分, 简单策略是 n 轮每次都对上/下半部分沿半径多切一刀使得多出希望的 q_i 的一段, 但如果两部分已经切出的和相等, 此时可以将两次沿半径的切分合并成沿直径的切分, 从而省下一步。

另一方面, 对于问题 A, 由于此时需要 $P = Q$ 此时阶段 2 其一定执行恰好 $n - 2m$ 步, 考虑证明假设其存在一组阶段 1 花费 m 代价的方案, 一定可使问题 B 得到一组 $c = m - 1$ 的解, 方案是显然的: $\forall 1 \leq i \leq m$, 将两个 a_i 的第一个拆出的所有小块都加入 A , 再将第二个拆除的所有小块都加入 B , 显然这些操作后结束时 A 与 B 中和相等。

从而这两个问题是等价的 (得到了双向规约), 而问题 B 可考虑状压 DP 求解: 设 $dp_{S', sum}$ 表示从 $S = \{1, 2, \dots, n\}$ 出发, 操作到 $S = S'$ 且目前 A 集合内 q_i 之和为 sum 的所有方案, 前面的步骤中 A 和 B 集合元素和相等的最大次数。注意这样设计状态其实也知道目前 B 集合内 q_i 之和, 从而能判断是否此时 A 和 B 集合元素和相等, 转移考虑枚举一个 $i \in S$ 考虑放到 A 还是 B 中即可。

时间复杂度为 $\Theta(2^n n \cdot 360)$ (这里只是为了表现出需要乘题面中圆环长度这一常数), 根据 DP 数组显然也能还原出问题 B 的一组最优解, 进而根据上述规约也能得到问题 A 的一组最优解, 并进一步得到原问题的一组最优解 (记得考虑全沿半径切的情况)。

接下来考虑 $\sum_{i=1}^n q_i$ 未必是 360 的情况, 考虑略有不同的问题 B':

- 初始下标集 $S = \{1, 2, \dots, n\}$, $A = B = \emptyset$ 。
- 进行 n 轮操作, 每轮选 $i \in S$, 然后将 i 从 S 删去, 并向 A 或 B 添加 i 。
- 结束时, 必须 $\sum_{i \in A} q_i \leq 180, \sum_{i \in B} q_i \leq 180$ 。
- 设前 $n - 1$ 轮操作后, $\sum_{i \in A} q_i = \sum_{i \in B} q_i$ 发生 c 次, 可对问题 A 构造出代价为 $n + 1 - [\sum_{i \in A} q_i = 180] - [\sum_{i \in B} q_i = 180] - c$ 的方案。

最后一点的论述是类似的, 区别在于目标是 $P = Q$ 时, 上下部分均全部用完, 因此上下部分的最后一刀都永远可以省下来, 而对于一般情况, 则未必可行。

出乎意料地, 对于问题 A 的最优解, 其仍然必定能对应到问题 B' 的一组最优解。考虑问题 B' 相当于假设上下半部分有用的弧都构成一段前缀, 即要么上 (或下) 半部分所有切出来的弧都被用上了, 要么只有最后一段没被用上, 而可以证明确实存在一组最优解满足该性质。

证明: 该性质可被重写为, 问题 A 存在一组最优解, 仅有阶段 1 的一个下标 $1 \leq i \leq m$ 满足, a_i 元素在被复制两份后, 其在阶段 2 的进一步切分不是每一块都被用上了 (上下部分对应的 a_i 都允许不被完全用完)。一个基本的性质是, 阶段 1 结束后可重集内的每个元素 x , 可假设其在阶段 2 的进一步拆分中, 至多一块被浪费。进一步假设存在 $1 \leq i \neq j \leq m$ 使得上部分的 a_i, a_j 均在进一步拆分中有一块被浪费, 此时在阶段 1 中用 $a_i + a_j$ 而非分开的 a_i 以及 a_j 仍然能得到不劣的方案 (即将阶段 1 的将 2 个 $a_i + a_j$ 同时切成 2 个 a_i 和 2 个 a_j 这一步留到阶段 2 去做)。现在上/下部分的 a_i 均至多有一个在进一步切分中存在一块被浪费, 设上部分的是 a_i , 下部分的是 a_j 切 $i \neq j$, 可考虑交换上部分的 a_j 与下部分的 a_i 的切分方案, 使得上部分的 a_i, a_j 均存在一块被浪费, 而下部分的 a_i, a_j 均全部用上, 此时根据前面的推理, 存在一部分同时两个大块被浪费则可以合并这两个大块, 答案不会变劣。每次调整都会减少阶段 1 的 m , 有限步内停止, 且停止时满足所需证性质。

问题 B' 可用与问题 B 类似的 DP 方法求解, 时间复杂度 $\Theta(2^n n \cdot 360)$ 。

